From Q-learning to Soft Actor-Critic: A Complete Derivation

Bellman optimality, temporal differences, DQN, maximum-entropy policy iteration, and continuous robot control

Foundation of Embodied AI ·Revised Lecture Notes

2026-10-18

Contents

Reading guide: six questions . . . 2
1. From MDPs to V and Q: why value functions matter . . . 2
2. From Bellman expectation to Bellman optimality . . . 3
3. Temporal-difference learning, SARSA, and tabular Q-learning . . . 4
4. DQN: scaling tabular Q-learning to large state spaces . . . 5
5. Why continuous actions push us toward actor-critic methods . . . 7
6. Maximum-entropy reinforcement learning . . . 7
7. Hard versus soft Bellman backups: from max to LogSumExp . . . 8
8. Soft policy improvement: why Boltzmann policies and KL divergence appear . . . 10
9. From soft policy iteration to modern SAC . . . 11
10. The complete SAC critic: target networks, twin Q, and TD regression . . . 12
11. The complete SAC actor: reparameterization, bounds, and log probabilities . . . 13
12. Automatic entropy tuning: why learn the temperature alpha? . . . 14
13. Putting together the complete modern SAC algorithm . . . 15
14. How the main algorithms relate . . . 16
15. Robot-control limits and common misconceptions . . . 17
16. The complete derivation in six equations . . . 17
References and further reading . . . 18

Course home ·简体中文 ·PDF

Item	Details
Applicability	Q-learning, TD/SARSA, DQN, soft Bellman equations, modern SAC, twin critics, reparameterization and automatic entropy tuning; instructional pseudocode
Documentation review date	2026-10-18
License	CC BY 4.0 ·Xiong Zeng and contributors

Reading guide: six questions

Verification scope: Document integration and rendering checks only; no training or robot validation. No experimental chapter is included. See the materials manifest for pinned source references. Original teaching content uses CC BY 4.0; third-party quotations, figures and code retain their original licenses. Citation alone does not grant relicensing rights.

> **Scope**. This self-contained tutorial reorganizes and extends the existing *Soft Actor-Critic (SAC): Complete Derivation and Design Motivations*. Starting with Q-learning, it derives modern continuous-action SAC by following the sequence **problem** → **mathematical principle** → **algorithmic design**. It contains no experimental section. A consistent convention for state/action values, entropy, and termination prevents accidental mixing of ordinary Bellman equations, soft Bellman equations, and their optimality counterparts.
>
> **Original course**: Course home ·Original Chinese notes ·Original English notes. This revision is included in the course repository; references to the original notes are pinned to the preceding commit for provenance.

- What does an action-value $Q(s, a)$ measure, and how does it differ from a state-value $V(s)$?
- Why can Q-learning learn without a dynamics model and update from behavior generated by a different policy?
- What problems do experience replay, target networks, and Double Q-learning address?
- Why is $\max_a Q(s, a)$ difficult for a high-dimensional robot action space?
- How does hard Bellman optimality connect to soft optimality, soft policy evaluation, and finally the SAC actor and critic?
- What specific problems are addressed by twin critics, reparameterization, and automatic entropy tuning?

Learning path: Bellman expectation → Bellman optimality → TD / Q-learning → DQN → continuous-action actor-critic → maximum-entropy RL → soft policy iteration → SAC.

Notation. s denotes the state; a, the action; $r(s, a)$, the expected one-step reward; $P(s' \mid s, a)$, the transition kernel; $\pi(a \mid s)$, a policy; and $\gamma \in [0, 1)$, the discount factor. The termination indicator $d = 1$ represents a true MDP termination, not merely an episode time-limit truncation. We write Q^π, V^π for the values of a fixed policy and Q^*, V^* for optimal values. In maximum-entropy settings we explicitly distinguish *soft* values where needed.

1. From MDPs to V and Q: why value functions matter

1.1 The reinforcement-learning objective

A discounted Markov decision process is

$$\mathcal{M} = (\mathcal{S}, \mathcal{A}, P, r, \gamma), \qquad 0 \le \gamma < 1.$$

A trajectory $\tau = (s_0, a_0, r_0, s_1, ...)$ has discounted return

$$G_t = \sum_{k=0}^{\infty} \gamma^k r_{t+k}, \qquad J(\pi) = \mathbb{E}_{\tau\sim\pi}[G_0].$$

The initial-state distribution is implicit in $J(\pi)$; in manipulation, it may vary with the initial object pose, joint configuration, or contact state. **An immediate reward is not an action value**: the former scores the present step; the latter also accounts for consequences far into the future.

1.2 State values versus action values

For a fixed policy π, define

$$\boxed{V^\pi(s) = \mathbb{E}_\pi[G_t \mid s_t = s], \qquad Q^\pi(s, a) = \mathbb{E}_\pi[G_t \mid s_t = s, a_t = a].}$$

$V^\pi(s)$ asks, "What is the expected future return from this state if we follow π?" $Q^\pi(s,a)$ asks, "What if we first take the specified action a, then follow π?" The Q-function directly supports comparisons between actions.

By the law of total expectation, for discrete actions,

$$\boxed{V^\pi(s) = \sum_a \pi(a \mid s) Q^\pi(s,a).}$$

For continuous actions replace the sum by $\int_{\mathcal{A}} \pi(a \mid s) Q^\pi(s,a)\, da$. A grasp can yield zero immediate reward but still have high Q-value if it puts the object in a state where subsequent lifting succeeds.

1.3 Bellman expectation: turning a long horizon into one step

Using $G_t = r_t + \gamma G_{t+1}$ and the Markov property,

$$\boxed{Q^\pi(s,a) = r(s,a) + \gamma \mathbb{E}_{s' \sim P(\cdot|s,a)}[V^\pi(s')].}$$

Substituting $V^\pi(s') = \mathbb{E}_{a' \sim \pi}[Q^\pi(s',a')]$ gives

$$\boxed{Q^\pi(s,a) = r(s,a) + \gamma \mathbb{E}_{s',a' \sim P,\pi}[Q^\pi(s',a')].}$$

This is the **Bellman expectation equation**: it evaluates a *given* policy, without maximizing over actions. The state-value form is $V^\pi(s) = \mathbb{E}_{a \sim \pi, s' \sim P}[r(s,a) + \gamma V^\pi(s')]$.

1.4 Policy evaluation and improvement

Classical policy iteration alternates between two operations. **Evaluation**: $\pi_k \mapsto Q^{\pi_k}$ estimates how good the current policy is. **Improvement**: $\pi_{k+1}(s) \in \arg\max_a Q^{\pi_k}(s,a)$ chooses a greedier policy. SAC preserves this two-step structure but replaces greedy improvement by entropy-regularized improvement of a stochastic policy.

2. From Bellman expectation to Bellman optimality

2.1 Optimal values and the optimality equations

Define $V^*(s) = \sup_\pi V^\pi(s)$ and $Q^*(s,a) = \sup_\pi Q^\pi(s,a)$. For a finite discrete action set, when the maximum exists,

$$\boxed{V^*(s) = \max_a Q^*(s,a),}$$

$$\boxed{Q^*(s,a) = r(s,a) + \gamma \mathbb{E}_{s' \sim P}[\max_{a'} Q^*(s',a')].}$$

The second equation is **Bellman optimality**. In contrast to policy evaluation, it inserts a maximization over the next action, rather than averaging according to a prescribed policy. It characterizes the optimal action-value function as a fixed point.

2.2 What is the Bellman operator buying us?

Define

$$(\mathcal{T}^* Q)(s,a) = r(s,a) + \gamma \mathbb{E}_{s'} \left[\max_{a'} Q(s',a') \right].$$

Under familiar bounded-reward discounted-MDP assumptions, the fact that $|\max_i x_i - \max_i y_i| \le \max_i |x_i - y_i|$ implies

$$\boxed{\|\mathcal{T}^*Q_1 - \mathcal{T}^*Q_2\|_\infty \le \gamma \|Q_1 - Q_2\|_\infty.}$$

Because $\gamma < 1$, the operator is a contraction on the relevant bounded-function space, with a unique fixed point Q^*. Exact value iteration $Q_{k+1} = \mathcal{T}^*Q_k$ converges. **This is not a convergence theorem for general neural-network Q-learning**, whose stochastic approximation and changing function class introduce further difficulties.

2.3 A two-action example

Consider two grasp choices with zero immediate reward. The optimal value of the state reached by the left grasp is 8, versus 10 for the right grasp. With $\gamma = 0.9$,

$$Q^*(s, a_L) = 0 + 0.9 \times 8 = 7.2, \qquad Q^*(s, a_R) = 0 + 0.9 \times 10 = 9.$$

Q prefers the right grasp even though neither action yields an immediate reward. This is why one-step reward maximization is insufficient for long-horizon manipulation.

3. Temporal-difference learning, SARSA, and tabular Q-learning

3.1 Monte Carlo versus TD: learning before an episode ends

Monte Carlo (MC) methods use a completed return G_t as a training target. Temporal-difference (TD) methods use a one-step reward plus an estimate of the next state's value. For policy evaluation, TD(0) uses

$$\delta_t^V = r_t + \gamma(1 - d_t)V(s_{t+1}) - V(s_t),$$

$$V(s_t) \leftarrow V(s_t) + \eta_t \delta_t^V.$$

TD can update before an episode ends, but its target **bootstraps** from another imperfect estimate. This introduces approximation-dependent bias and error propagation. It is an oversimplification to claim that the complete MC–TD distinction is always just "unbiased versus biased"; the estimand, sampling process, and approximation assumptions matter.

3.2 SARSA evaluates the policy being executed

SARSA uses the actual next action a_{t+1} sampled from the behavior policy:

$$\boxed{y_t^{\text{SARSA}} = r_t + \gamma(1 - d_t)Q(s_{t+1}, a_{t+1}).}$$

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \eta_t(y_t^{\text{SARSA}} - Q(s_t, a_t)).$$

When a_{t+1} follows the same ε-greedy policy being improved, standard SARSA is **on-policy TD control**: its target reflects the exploratory policy's next action.

3.3 Q-learning can explore while its target is greedy

Tabular Watkins Q-learning uses

$$\boxed{y_t^Q = r_t + \gamma(1 - d_t) \max_{a'} Q_t(s_{t+1}, a'),}$$

$$\boxed{Q_{t+1}(s_t, a_t) = Q_t(s_t, a_t) + \eta_t \left[y_t^Q - Q_t(s_t, a_t) \right].}$$

The bracketed quantity is the TD error δ_t: **new estimate = old estimate + step size × prediction error**. An ε-greedy behavior policy can take a greedy action with probability $1 - \varepsilon$ and select a uniform exploratory action with probability ε (conventions regarding overlap can vary). The **behavior policy** β generates transitions. The Q-learning target uses greedy maximization, corresponding to a different **target policy**. Thus tabular Q-learning is **off-policy**.

For a numerical illustration, suppose $Q(s, a) = 4$, $r = 1$, the highest next-state Q equals 6, $\gamma = 0.9$, and $\eta = 0.2$. Then

$$y = 1 + 0.9 \times 6 = 6.4, \qquad Q_{\text{new}} = 4 + 0.2(6.4 - 4) = 4.48.$$

3.4 Comparing the three TD targets

Algorithm	One-step target (nonterminal case)	Next-action rule	Standard data regime
SARSA	$r + \gamma Q(s', a')$	Sample a' from behavior policy	On-policy
Expected SARSA	$r + \gamma \mathbb{E}_{a' \sim \pi} Q(s', a')$	Expectation under chosen target policy	May be on/off-policy
Q-learning	$r + \gamma \max_{a'} Q(s', a')$	Greedy optimality backup	Off-policy

Important: Replay buffers are *not what makes Q-learning off-policy*. Tabular Q-learning can be off-policy even when each sample is processed immediately. A replay buffer is a separate mechanism for reusing transitions and altering their sampling correlations.

3.5 When does tabular Q-learning converge?

The classical result covers finite tabular state/action settings with a stationary MDP, bounded rewards, sufficiently repeated visits, and appropriate step sizes (e.g., Robbins–Monro-type conditions), among other assumptions. It establishes convergence to Q^* in the specified setting. **It does not automatically cover nonlinear function approximation, inadequate coverage, changing environments, or arbitrary combinations of off-policy training and bootstrapping.** This distinction matters when we move to DQN and SAC. See Watkins and Dayan (1992).

4. DQN: scaling tabular Q-learning to large state spaces

4.1 From a Q-table to neural TD regression

A high-dimensional observation space makes a Q-table impractical. Introduce a neural approximator $Q_\phi(s, a)$ and the sample target

$$y = r + \gamma(1 - d) \max_{a'} Q_{\bar{\phi}}(s', a').$$

Train by minimizing

$$\boxed{J_Q(\phi) = \frac{1}{2} \mathbb{E}_{(s,a,r,s',d) \sim \mathcal{D}} \left[\left(Q_\phi(s, a) - \text{stopgrad}(y) \right)^2 \right].}$$

This has the same *regression structure* as SAC's critic loss: both fit Q to a bootstrap-based TD target. The crucial differences concern which action is evaluated in the next state, whether entropy is included, and whether a separately parameterized actor is learned.

4.2 Why experience replay helps

Consecutive observations are strongly correlated. Updating a neural Q-function only on the most recent transition can make gradients reflect a narrow and changing slice of experience. A replay buffer,

$$\mathcal{D} = \{(s_t, a_t, r_t, s_{t+1}, d_t)\}_{t=1}^{N},$$

supports minibatch sampling from earlier interactions, weakens short-range temporal dependence, and permits multiple updates from a transition. **Replay does not manufacture coverage**: a dataset consisting entirely of near-identical grasps does not tell us what a never-tried grasp would accomplish.

4.3 Why target networks help

If the prediction $Q_\phi(s, a)$ and target $r + \gamma \max_{a'} Q_\phi(s', a')$ change aggressively together, the supervised target is continuously moving. DQN introduces a delayed target Q-network $Q_{\bar{\phi}}$. It may be updated periodically with $\bar{\phi} \leftarrow \phi$, or, in other actor-critic algorithms, by Polyak averaging:

$$\boxed{\bar{\phi} \leftarrow (1-\tau)\bar{\phi} + \tau\phi, \qquad 0 < \tau \ll 1.}$$

Periodic hard copying is characteristic of classical DQN; small-step Polyak updates are common in modern continuous-control methods such as SAC and TD3. These stabilizers limit target drift, but do not remove the errors inherent in approximation.

4.4 How maximization amplifies value overestimation

Suppose $\widehat{Q}(s, a) = Q(s, a) + \epsilon_a$, with unbiased estimation errors for individual actions. Maximizing noisy estimates can still create positive bias:

$$\mathbb{E}\left[\max_a (Q_a + \epsilon_a)\right] \geq \max_a \mathbb{E}[Q_a + \epsilon_a] = \max_a Q_a.$$

The inequality follows from the convexity of the maximum (Jensen's inequality). Larger action sets and noisier estimates can make this selection effect more consequential.

4.5 Double Q-learning, Double DQN, and SAC's twin critics differ

The motivating principle behind Double Q-learning is to **separate action selection from action evaluation**, thereby reducing the bias that arises when both are performed using the same noisy estimator:

- **Tabular Double Q-learning**: two Q-tables alternately choose and evaluate greedy actions, with their roles switched during different updates.
- **Double DQN**: the online network selects the maximizing action, while the target network evaluates it:

$$\boxed{a^* = \arg\max_{a'} Q_\phi(s', a'), \qquad y_{\text{DDQN}} = r + \gamma(1-d)Q_{\bar{\phi}}(s', a^*).}$$

- **SAC's clipped double Q**: for an action a' sampled from the actor, the target uses $\min(Q_{\bar{\phi}_1}(s', a'), Q_{\bar{\phi}_2}(s', a'))$. This conservative estimator reduces some overoptimism but can underestimate value.

The three mechanisms are related **but their formulas and statistical biases are not identical**. See van Hasselt (2010) and Double DQN.

4.6 The deadly triad: why deep value learning can diverge

The combination of **function approximation, bootstrapping, and off-policy training** is often called the *deadly triad*. A Q-function with limited state-action support may propagate its own prediction errors into future targets. The actor or greedy selector may then exploit the inaccurate values, further concentrating the data or targets in unreliable regions. Experience replay, target networks, and twin critics alleviate some problems; they are not blanket convergence guarantees.

5. Why continuous actions push us toward actor-critic methods

5.1 The action maximum becomes an expensive inner optimization

Q-learning uses

$$\max_{a' \in \mathcal{A}} Q_\phi(s', a').$$

For a small discrete action set, we can enumerate all actions. Robot control actions are often continuous, $a \in \mathbb{R}^d$, e.g., many joint targets or torque commands. A learned $Q_\phi(s, a)$ is typically nonconvex in a. Solving a new high-dimensional optimization problem for every Bellman target is expensive and can be trapped at suboptimal local maxima.

5.2 An actor is an amortized optimizer of actions

Introduce a parameterized deterministic actor $a = \mu_\theta(s)$ and retain a critic $Q_\phi(s, a)$. A standard deterministic actor objective is

$$J_\mu(\theta) = -\mathbb{E}_{s \sim \mathcal{D}}[Q_\phi(s, \mu_\theta(s))],$$

$$\nabla_\theta J_\mu = -\mathbb{E}_s \left[\nabla_a Q_\phi(s, a)|_{a=\mu_\theta(s)} \nabla_\theta \mu_\theta(s) \right].$$

The actor **amortizes action optimization** into a forward pass, instead of repeatedly solving $\arg\max_a Q$ exactly. DDPG exemplifies off-policy deterministic actor-critic control. TD3 augments that family with twin critics, delayed actor updates, and target policy smoothing, among other stabilization choices.

5.3 Why does SAC adopt a stochastic actor?

Deterministic policies have no intrinsic conditional action entropy and often add separate exploratory noise. SAC instead parameterizes a stochastic conditional policy $\pi_\theta(a \mid s)$ and jointly rewards high expected Q and policy entropy:

$$\max_\pi \ \mathbb{E}_{a \sim \pi}[Q(s, a)] + \alpha \mathcal{H}(\pi(\cdot \mid s)).$$

This is a design choice, not a theorem that continuous control *must* have an explicit actor: discretization, sampling-based optimization, and implicit policy methods also exist. SAC uses a tractable, differentiable actor to make continuous soft policy improvement efficient.

6. Maximum-entropy reinforcement learning

6.1 Policy entropy and temperature

For discrete actions, $\mathcal{H}(\pi) = -\sum_a \pi(a \mid s) \log \pi(a \mid s)$. For continuous actions, differential entropy is

$$\boxed{\mathcal{H}(\pi(\cdot \mid s)) = -\int_{\mathcal{A}} \pi(a \mid s) \log \pi(a \mid s)\, da = \mathbb{E}_{a \sim \pi}[-\log \pi(a \mid s)].}$$

Unlike discrete Shannon entropy, differential entropy may be negative and depends on the units of the action coordinates. For a one-dimensional Gaussian, $\mathcal{N}(\mu, \sigma^2)$, $\mathcal{H} = \frac{1}{2}\log(2\pi e\sigma^2)$.

With temperature $\alpha \geq 0$, maximum-entropy RL optimizes

$$\boxed{J_{\mathrm{ME}}(\pi) = \mathbb{E}_{\tau\sim\pi}\left[\sum_{t=0}^{\infty}\gamma^t(r_t - \alpha\log\pi(a_t \mid s_t))\right].}$$

A larger α puts more weight on entropy; the $\alpha \to 0^+$ limit approaches ordinary return maximization. The entropy contribution depends on the **current policy**; it is not a policy-independent environmental reward.

6.2 A consistent convention for soft Q and soft V

To align the derivation with modern SAC targets, define

$$V^{\pi}_{\mathrm{soft}}(s) = \mathbb{E}_{\pi}\left[\sum_{t=0}^{\infty}\gamma^t(r_t - \alpha\log\pi(a_t \mid s_t)) \mid s_0 = s\right],$$
$$Q^{\pi}_{\mathrm{soft}}(s,a) = \mathbb{E}_{\pi}\left[r_0 + \sum_{t=1}^{\infty}\gamma^t(r_t - \alpha\log\pi(a_t \mid s_t)) \mid s_0 = s, a_0 = a\right].$$

The **current action's entropy is included in soft V**; because the current action is fixed, it is **excluded from soft Q** and begins at the next time step. Consequently,

$$\boxed{V^{\pi}_{\mathrm{soft}}(s) = \mathbb{E}_{a\sim\pi}\left[Q^{\pi}_{\mathrm{soft}}(s,a) - \alpha\log\pi(a \mid s)\right],}$$

$$\boxed{Q^{\pi}_{\mathrm{soft}}(s,a) = r(s,a) + \gamma\mathbb{E}_{s'}[V^{\pi}_{\mathrm{soft}}(s')].}$$

Combining them gives the **soft Bellman expectation equation**:

$$\boxed{Q^{\pi}_{\mathrm{soft}}(s,a) = r(s,a) + \gamma\mathbb{E}_{s',a'\sim P,\pi}\left[Q^{\pi}_{\mathrm{soft}}(s',a') - \alpha\log\pi(a' \mid s')\right].}$$

Compared with the ordinary expectation equation from Section 1, the structural change is the additional $-\alpha\log\pi$ term inside the next-state soft value.

7. Hard versus soft Bellman backups: from max to LogSumExp

7.1 Optimal soft value for a finite discrete action set

Fix state s, assume finite actions and $\alpha > 0$, and suppose a candidate Q-function is given. Maximize over $p_a \geq 0$, $\sum_a p_a = 1$,

$$F(p) = \sum_a p_a Q(s,a) - \alpha\sum_a p_a\log p_a.$$

With multiplier λ for the normalization constraint, stationarity requires

$$\frac{\partial}{\partial p_a}\left(F(p) + \lambda(\sum_a p_a - 1)\right) = Q(s,a) - \alpha(\log p_a + 1) + \lambda = 0.$$

Thus the optimal policy is a **Boltzmann distribution**:

$$\boxed{p_a^* = \frac{\exp(Q(s,a)/\alpha)}{\sum_b \exp(Q(s,b)/\alpha)}.}$$

Substitution produces **LogSumExp**:

$$\boxed{\max_p F(p) = \alpha \log \sum_a \exp\left(Q(s,a)/\alpha\right).}$$

Therefore, the optimal soft Bellman operator for finite discrete actions is

$$\boxed{(\mathcal{T}^*_{\text{soft}} Q)(s,a) = r(s,a) + \gamma \mathbb{E}_{s'}\left[\alpha \log \sum_{a'} e^{Q(s',a')/\alpha}\right].}$$

The Q here follows the soft-Q convention in Section 6, which excludes the current action's entropy. This equation is not obtained by indiscriminately adding LogSumExp to an unrelated definition of conventional Q-value.

7.2 The zero-temperature limit recovers the maximum

Let $M = \max_i q_i$ and let n be the number of discrete actions. Then

$$\alpha \log \sum_i e^{q_i/\alpha} = M + \alpha \log \sum_i e^{(q_i - M)/\alpha}.$$

Because $1 \le \sum_i e^{(q_i-M)/\alpha} \le n$,

$$\boxed{M \le \alpha \log \sum_i e^{q_i/\alpha} \le M + \alpha \log n,}$$

which proves

$$\boxed{\lim_{\alpha \to 0^+} \alpha \log \sum_i e^{q_i/\alpha} = \max_i q_i.}$$

This is the central conceptual bridge: **the hard maximum is the zero-temperature limit of an entropy-regularized soft maximum**. With nonzero temperature, multiple high-value actions can retain meaningful probability.

7.3 Continuous-action integrals and their assumptions

For continuous actions the ideal soft-optimal value takes the form

$$V^*_{\text{soft}}(s) = \alpha \log \int_{\mathcal{A}} \exp(Q^*_{\text{soft}}(s,a)/\alpha)\, da.$$

This expression requires a finite normalization integral and depends on the base measure for action densities. Rescaling coordinates changes differential entropy and normalization. For high-dimensional nonconvex Q-functions, the integral is generally intractable. **SAC does not directly evaluate this integral as part of its standard learning loop**: it trains an actor to approximate the corresponding high-value stochastic policy.

7.4 Do not confuse soft optimality with soft policy evaluation

Equation / operator	What is being solved?	Treatment of the next action
$\mathcal{T}^{\pi}Q$	Evaluate a fixed policy π	$\mathbb{E}_{a'\sim\pi}$
$\mathcal{T}^{*}Q$	Unregularized optimal control	$\max_{a'}$
$\mathcal{T}^{*}_{\text{soft}}Q$	Entropy-regularized optimal control	$\alpha \log \sum e^{Q/\alpha}$ (discrete)
SAC critic target	Approximate soft evaluation of the **current actor**	Sample $a' \sim \pi_\theta$ and subtract $\alpha \log \pi_\theta$

This table is critical. SAC does **not** merely substitute a tractable explicit LogSumExp for Q-learning's max. It performs approximate *soft policy iteration*, with an actor representing the policy whose value is estimated by its critic.

8. Soft policy improvement: why Boltzmann policies and KL divergence appear

8.1 From the ideal distribution to a trainable policy

Fix a state s and the soft Q of an old policy: $Q_{\text{old}}(s,a) = Q^{\pi_{\text{old}}}_{\text{soft}}(s,a)$. The ideal improved policy solves

$$\pi_{\text{new}} \in \arg\max_{\pi} \left\{ \mathbb{E}_{a\sim\pi}[Q_{\text{old}}(s,a)] + \alpha \mathcal{H}(\pi(\cdot \mid s)) \right\}.$$

With suitable integrability and density assumptions, the continuous-action solution remains the Boltzmann policy

$$\boxed{\pi_B(a \mid s) = \frac{\exp(Q_{\text{old}}(s,a)/\alpha)}{Z_{\text{old}}(s)}, \quad Z_{\text{old}}(s) = \int_{\mathcal{A}} e^{Q_{\text{old}}(s,a')/\alpha}\, da'.}$$

However, evaluating $Z_{\text{old}}(s)$ and sampling exactly from π_B are difficult in high-dimensional action spaces.

8.2 Reverse KL removes the intractable normalizer

Introduce a parameterized actor π_θ and fit it to the Boltzmann policy by expanding $D_{\text{KL}}(\pi_\theta \| \pi_B)$:

$$\begin{aligned} D_{\text{KL}}(\pi_\theta \| \pi_B) &= \mathbb{E}_{a\sim\pi_\theta}\left[\log \pi_\theta(a \mid s) - \log \pi_B(a \mid s)\right] \\ &= \mathbb{E}_{a\sim\pi_\theta}\left[\log \pi_\theta(a \mid s) - Q_{\text{old}}(s,a)/\alpha\right] + \log Z_{\text{old}}(s). \end{aligned}$$

Since $\log Z_{\text{old}}$ does not depend on θ, minimizing this KL divergence is equivalent to

$$\boxed{\min_{\theta} \mathbb{E}_{a\sim\pi_\theta}\left[\alpha \log \pi_\theta(a \mid s) - Q_{\text{old}}(s,a)\right].}$$

This is the theoretical prototype of SAC's actor loss. The direction matters: we are using $D_{\text{KL}}(\pi_\theta \| \pi_B)$, not $D_{\text{KL}}(\pi_B \| \pi_\theta)$. These two projections can have different approximation behavior.

8.3 Improvement under exact soft policy iteration

Assume the old policy belongs to the admissible policy class and the new policy exactly solves the statewise KL projection. Optimality then implies

$$\mathbb{E}_{a\sim\pi_{\text{new}}}[Q^{\pi_{\text{old}}}_{\text{soft}}(s,a) - \alpha \log \pi_{\text{new}}(a \mid s)] \geq V^{\pi_{\text{old}}}_{\text{soft}}(s).$$

Define the soft policy-evaluation operator

$$(\mathcal{T}^{\pi}_{\text{soft}}Q)(s,a) = r(s,a) + \gamma \mathbb{E}_{s',a' \sim P,\pi}[Q(s',a') - \alpha \log \pi(a' \mid s')].$$

Under suitable boundedness assumptions, this operator is monotone and a γ-contraction. Repeated application under the improved policy yields

$$\boxed{Q^{\pi_{\text{new}}}_{\text{soft}}(s,a) \geq Q^{\pi_{\text{old}}}_{\text{soft}}(s,a).}$$

This statement concerns **exact policy evaluation and exact improvement**. It does not establish global monotonic improvement or convergence for practical SAC with finite replay data, neural approximators, and stochastic optimization.

9. From soft policy iteration to modern SAC

9.1 Two conceptual operations, represented by neural networks

We now have a complete policy-iteration cycle:

1. **Soft policy evaluation**: learn the current policy's soft Q-function through the soft Bellman equation.
2. **Soft policy improvement**: update the actor toward higher-Q, entropy-regularized actions.

With neural function approximation,

$$\boxed{\text{Actor: } \pi_\theta(a \mid s), \qquad \text{Critics: } Q_{\phi_1}(s,a),\ Q_{\phi_2}(s,a).}$$

The critic maps state-action pairs to scalar value estimates. The actor maps states to conditional action distributions. Rewards train the critics, and Q-value gradients then inform actor improvement. In model-free SAC, one does not normally differentiate the environmental reward through the physical simulator directly into the actor.

9.2 Why SAC can reuse data collected by older policies

For a stationary MDP, the conditional transition law $P(s' \mid s,a)$ does not depend on which behavior policy β happened to generate the pair (s,a). A stored transition (s,a,r,s',d) therefore remains a valid sample of environmental dynamics even when the actor has changed. However, the **next action in the target** is sampled from the **current actor**:

$$y = r + \gamma(1-d)\left[Q(s',a') - \alpha \log \pi_\theta(a' \mid s')\right], \qquad a' \sim \pi_\theta(\cdot \mid s').$$

Quantity	Source	Recomputed under the current actor?
(s,a,r,s',d)	Historical replay transition	No
a'	Sample $\pi_\theta(\cdot \mid s')$	Yes
$\log \pi_\theta(a' \mid s')$	Current actor density	Yes
$Q(s',a')$	Target critic	Yes

Replay improves sample reuse, but not every historical distribution yields a reliable Bellman update. **Insufficient state-action coverage, critic extrapolation, function approximation, and bootstrap error** remain serious issues. Actor updates often use states from the replay distribution rather than the current policy's exact discounted occupancy measure, which is a further approximation.

9.3 How this differs from on-policy PPO

PPO uses actions sampled from an old policy and the likelihood ratio $\rho_t(\theta) = \pi_\theta(a_t \mid s_t)/\pi_{\text{old}}(a_t \mid s_t)$. Standard PPO typically performs a limited number of optimization epochs on recent rollouts; it does not treat arbitrary ancient rollout actions as if they were sampled from the current actor. SAC instead uses off-policy Bellman evaluation and actor improvement and does not require PPO's probability-ratio clipping. This is a distinction between standard algorithmic settings, not a claim that no PPO-related method could ever use older experience.

10. The complete SAC critic: target networks, twin Q, and TD regression

10.1 From the soft Bellman equation to a sampled target

Maintain two critics Q_{ϕ_i} and two slowly moving target critics $Q_{\bar{\phi}_i}$. For (s, a, r, s', d) from replay, sample $a' \sim \pi_\theta(\cdot \mid s')$ and define

$$\boxed{y = r + \gamma(1-d)\left[\min_{j=1,2} Q_{\bar{\phi}_j}(s', a') - \alpha \log \pi_\theta(a' \mid s')\right].}$$

Each critic minimizes a regression loss:

$$\boxed{J_{Q_i}(\phi_i) = \frac{1}{2}\mathbb{E}_{\mathcal{D}}\left[(Q_{\phi_i}(s,a) - \text{stopgrad}(y))^2\right], \quad i = 1, 2.}$$

The target is a stochastic one-sample approximation of the Bellman expectation. Gradients must not flow through the target branch; otherwise the operation ceases to match this standard semi-gradient TD regression construction.

10.2 Why use the minimum of two critics?

An actor tends to seek actions with high estimated Q-values. If high Q-values arise from approximation errors, these errors can be exploited by the actor and propagated by bootstrapping. SAC uses $\min_j Q_{\bar{\phi}_j}(s', a')$ to introduce a conservative estimate for a *given* next action. This reduces certain kinds of positive bias, but **does not make estimates unbiased**. Correlated errors can persist, and taking the minimum may introduce negative bias.

Crucially, classical Q-learning's $\max_{a'}$ is an **action-space optimality backup**. SAC's $\min_j$ is a **reduction over critic estimators**. They operate on different indices and solve different problems.

10.3 True termination versus time-limit truncation

If $d = 1$ represents genuine termination, the target reduces to $y = r$. If an environment truncates a continuing episode only because a step limit was reached (`truncated=True`), bootstrapping should usually be retained. Conflating `terminated` and `truncated` can systematically undervalue states near artificial episode boundaries. For truly finite-horizon problems, one should define the terminal condition and include remaining time in the Markov state when necessary.

10.4 Slowly updating the target critics

A typical Polyak update is

$$\boxed{\bar{\phi}_j \leftarrow (1-\tau)\bar{\phi}_j + \tau\phi_j, \qquad 0 < \tau \ll 1.}$$

A common illustrative setting is $\tau = 0.005$, but no single value is universally optimal. A slow-moving target limits short-term target drift; it cannot compensate for flawed reward scaling, inadequate coverage, or unstable action representations.

11. The complete SAC actor: reparameterization, bounds, and log probabilities

11.1 From KL projection to the actor objective

Using replay states for approximate policy improvement, a common actor objective is

$$\boxed{J_\pi(\theta) = \mathbb{E}_{\substack{s\sim\mathcal{D} \\ a\sim\pi_\theta(\cdot|s)}} \left[\alpha \log \pi_\theta(a \mid s) - \min_{j=1,2} Q_{\phi_j}(s,a)\right].}$$

The first term favors appropriate entropy, and the second favors high-value actions. During actor optimization one usually freezes the critic **parameters**, but preserves $\nabla_a Q_\phi(s,a)$ to transmit an improvement direction to the actor. Some implementations update the actor against just Q_{ϕ_1} rather than $\min_j Q_{\phi_j}$; this is an implementation convention to verify, not a mathematical identity.

11.2 The reparameterization trick

Writing only $a \sim \pi_\theta(a \mid s)$ obscures how derivatives flow through random sampling. Let $\epsilon \sim \mathcal{N}(0, I)$ be independent of the actor parameters, and let the actor output $\mu_\theta(s)$ and $\sigma_\theta(s)$:

$$\boxed{u_\theta(s,\epsilon) = \mu_\theta(s) + \sigma_\theta(s) \odot \epsilon,}$$

$$\boxed{a_\theta(s,\epsilon) = \tanh u_\theta(s,\epsilon).}$$

Then $a_\theta = f_\theta(s,\epsilon)$ is differentiable with respect to θ, while the base noise distribution is fixed. The objective becomes

$$J_\pi(\theta) = \mathbb{E}_{s\sim\mathcal{D},\epsilon\sim\mathcal{N}(0,I)} \left[\alpha \log \pi_\theta(f_\theta(s,\epsilon) \mid s) - Q_\phi(s, f_\theta(s,\epsilon))\right].$$

Here Q_ϕ stands for whichever critic reduction is used in the actor update (one critic or the minimum of two). The derivative includes

$$-\underbrace{\nabla_a Q_\phi(s,a_\theta)}_{\text{action-value gradient}} \underbrace{\frac{\partial a_\theta}{\partial \theta}}_{\text{actor action Jacobian}},$$

plus the **total derivative** of $\alpha \log \pi_\theta(a_\theta \mid s)$: both explicit parameter dependence and implicit dependence through the sampled action matter. This pathwise gradient differs from the score-function gradient commonly used in PPO. It is not simply a distinction between deterministic and stochastic policies.

11.3 Correct log probability for a tanh-squashed Gaussian

For $u \sim \mathcal{N}(\mu_\theta(s), \operatorname{diag}\sigma_\theta^2(s))$ and $a = \tanh u$, the change-of-variables formula gives

$$\pi_\theta(a \mid s) = p_U(u \mid s) \left|\det \frac{\partial a}{\partial u}\right|^{-1}.$$

Because $\partial a_i / \partial u_i = 1 - \tanh^2 u_i$,

$$\boxed{\log \pi_\theta(a \mid s) = \log \mathcal{N}(u; \mu_\theta(s), \Sigma_\theta(s)) - \sum_{i=1}^{d} \log(1 - \tanh^2 u_i).}$$

Substituting $\log \mathcal{N}(a; \mu, \Sigma)$ is incorrect. For numerical stability near $\tanh u = \pm 1$, use

$$\log(1-\tanh^2 u) = 2(\log 2 - u - \mathrm{softplus}(-2u)).$$

11.4 Normalized versus physical robot actions

Suppose the actual executed action dimension is $a_i^{\mathrm{env}} = c_i + b_i \tanh u_i$, where

$$c_i = \frac{a_{\max,i} + a_{\min,i}}{2}, \qquad b_i = \frac{a_{\max,i} - a_{\min,i}}{2} > 0.$$

The log density **with respect to physical action coordinates** includes the additional $-\sum_i \log b_i$ term:

$$\log \pi_\theta(a^{\mathrm{env}} \mid s) = \log \mathcal{N}(u; \mu, \Sigma) - \sum_i \log\left[b_i(1-\tanh^2 u_i)\right].$$

An alternative, common convention defines all probabilities, entropy targets, and critic action inputs in normalized $[-1,1]^d$ coordinates and then applies a fixed environment-level action scaling. Both conventions are valid if applied consistently. The constant log-Jacobian shift affects numerical differential entropy and the interpretation of automatic temperature tuning.

12. Automatic entropy tuning: why learn the temperature alpha?

12.1 From fixed temperature to a target-entropy constraint

The effect of a fixed α varies with reward scaling and action dimension. Modern SAC can instead impose an average entropy requirement:

$$\max_\pi J_R(\pi) \quad \text{s.t.} \quad \bar{\mathcal{H}}(\pi) \geq \mathcal{H}_{\text{target}}.$$

The corresponding Lagrangian, in this simplified average-constraint representation, is

$$\mathcal{L}(\pi, \alpha) = J_R(\pi) + \alpha(\bar{\mathcal{H}}(\pi) - \mathcal{H}_{\text{target}}), \qquad \alpha \geq 0.$$

The primal-dual structure is $\max_\pi \min_{\alpha\geq 0} \mathcal{L}(\pi,\alpha)$. Original SAC derivations can use per-time-step entropy constraints; the average-entropy form here conveys the design motivation, but should not be assumed fully equivalent to all other constraint formulations.

12.2 The temperature objective and a sign check

With the actor held fixed, minimize

$$\boxed{J_\alpha(\alpha) = \mathbb{E}_{s\sim\mathcal{D}, a\sim\pi_\theta}\left[\alpha(-\log \pi_\theta(a \mid s) - \mathcal{H}_{\text{target}})\right].}$$

The derivative is $\partial J_\alpha / \partial \alpha = \bar{\mathcal{H}}(\pi) - \mathcal{H}_{\text{target}}$. If measured entropy is below target, this derivative is negative and gradient descent increases α; if entropy exceeds target, descent reduces α. **Negative numerical targets are perfectly possible for continuous differential entropy**; the intuition from nonnegative discrete entropy does not carry over unchanged.

12.3 Why parameterize alpha as exp(eta)?

Use $\eta = \log \alpha$ and $\alpha = e^\eta > 0$ to preserve positivity. Strict substitution yields

$$J_\eta(\eta) = -\mathbb{E}\left[e^\eta(\log \pi_\theta(a \mid s) + \mathcal{H}_{\text{target}})\right].$$

Many implementations instead use a surrogate with a different gradient scale but the same interior stationary condition:

$$\boxed{J_\eta^{\text{sur}}(\eta) = -\mathbb{E}\left[\eta(\log \pi_\theta(a \mid s) + \mathcal{H}_{\text{target}})\right].}$$

The log probability must be detached during the temperature update to avoid accidentally optimizing the actor through the temperature objective. The common heuristic $\mathcal{H}_{\text{target}} = -d$ for d normalized action dimensions is **not a theorem**. Correlated joints, high-dimensional hands, and constrained actions require care when interpreting it.

13. Putting together the complete modern SAC algorithm

13.1 Components and three losses

Component	Symbol	Role
Stochastic actor	$\pi_\theta(a \mid s)$	Samples valuable continuous actions with nonzero entropy
Twin critics	Q_{ϕ_1}, Q_{ϕ_2}	Estimate soft Q and reduce overoptimism
Target critics	$Q_{\bar{\phi}_1}, Q_{\bar{\phi}_2}$	Provide slowly changing TD targets
Replay buffer	$\mathcal{D}$	Store and reuse old transitions off-policy
Temperature	$\alpha = e^\eta$	Adjust entropy weight to a target

The optimization problems are the two critic regressions $J_{Q_1} + J_{Q_2}$, the actor loss J_π, and the temperature loss J_η. Their interaction is best understood as **evaluate the actor → improve the actor → tune entropy → slowly track target values**.

13.2 The full sequence of one SAC update

1. Interact with the environment using the actor (possibly with random warm-up exploration) and save $(s, a, r, s', \text{terminated}, \text{truncated})$.
2. Sample a replay minibatch. At each s', resample a' from the current actor; use target critics and α to construct the soft Bellman target y with target-branch gradients disabled.
3. Regress each $Q_{\phi_i}(s, a)$ onto the same target y.
4. At replay states s, resample $\tilde{a}$ from the current actor. Update it using $\alpha \log \pi_\theta(\tilde{a} \mid s) - \min_i Q_{\phi_i}(s, \tilde{a})$, keeping critic parameters fixed but the gradient with respect to the input action enabled.
5. Update $\log \alpha$ using detached policy log probabilities to adjust the entropy pressure.
6. Update target critics by Polyak averaging. Continue data collection and optimization, potentially using more than one gradient step per environment step.

The following is **mathematical pseudocode**, not executable Python and not an experimental protocol:

```
Initialize actor pi_theta; critics Q_phi1,Q_phi2
Initialize target critics Q_barphi1,Q_barphi2; replay D; eta=log(alpha)
Repeat:
    Collect environment transition and append to D
    For each gradient update:
        Sample (s,a,r,s_next,terminated,truncated) from D
        a_next ~ pi_theta(. | s_next)
        y = r + gamma*(1-terminated)*(
                min_i Q_barphi_i(s_next,a_next) - alpha*log pi_theta(a_next|s_next))
        Update each Q_phi_i by gradient of (Q_phi_i(s,a)-stopgrad(y))^2
        a_pi ~ pi_theta(. | s)  # reparameterized sample
        Update actor by gradient of alpha*log pi_theta(a_pi|s)-min_i Q_phi_i(s,a_pi)
        Update eta by entropy dual loss (detach log probability)
        barphi_i <- (1-tau)*barphi_i + tau*phi_i
```

Respecting `terminated` versus `truncated`, normalized action conventions, the tanh density Jacobian, frozen critic

weights during actor updates, and replay coverage are essential implementation details rather than optional embellishments.

13.3 Original SAC v1 versus modern SAC: is V explicitly learned?

The original SAC version additionally fit a state-value network $V_\psi(s)$:

$$J_V(\psi) = \frac{1}{2}\mathbb{E}_s\left[\left(V_\psi(s) - \text{stopgrad}(\min_i Q_{\phi_i}(s,\tilde{a}) - \alpha \log \pi_\theta(\tilde{a} \mid s))\right)^2\right],$$

with Q-target $y_Q = r+\gamma(1-d)V_{\bar{\psi}}(s')$. The modern widely used variant removes the explicit V-network and constructs Q-targets directly from the target Q-networks and the current actor's next-action log probability. Thus implementations often contain an actor, two critics, two target critics, and a learned temperature. **Removing the V-network does not remove the soft V concept from the mathematics**.

14. How the main algorithms relate

Method	Typical value backup	Actor / action selection	Data regime	Main design principle
SARSA	$Q(s',a')$ for next behavior action	Behavior policy	On-policy (standard)	Evaluate the policy being executed
Q-learning	$\max_{a'} Q(s',a')$	Greedy + exploration	Off-policy	Bellman optimality
DQN	Neural $\max_{a'} Q_{\bar{\phi}}$	Discrete argmax	Off-policy	Replay and target networks
Double DQN	Online selection, target evaluation	Discrete argmax	Off-policy	Reduce max-induced overestimation
DDPG	Target Q at deterministic actor action	Deterministic actor + exploration noise	Off-policy	Continuous-action policy gradient
TD3	Twin Q with conservative target	Delayed deterministic actor	Off-policy	Reduce approximation error
SAC	Soft Bellman with twin critics	Stochastic actor + entropy	Off-policy	Soft policy iteration
PPO	Typically V and advantage estimates	Stochastic policy likelihood-ratio updates	On-policy (standard)	Clipped policy objective

14.1 The most important Q-learning versus SAC distinction

Classical Q-learning attempts to directly learn optimal Q and uses $\max_{a'} Q(s',a')$ in its target. SAC's critic estimates the **current stochastic actor's soft Q**, while the actor performs entropy-regularized improvement:

$$\boxed{\begin{aligned}&\text{Q-learning: } y = r + \gamma \max_{a'} Q(s',a'),\\ &\text{SAC: } y = r + \gamma\left[\min_j Q_{\bar{\phi}_j}(s',a') - \alpha \log \pi(a' \mid s')\right], \quad a' \sim \pi.\end{aligned}}$$

We omit the true-termination factor for readability; practical implementations multiply the bootstrap term by $1-d$. In SAC, $\min_j$ does not minimize over actions. It chooses the lower of several estimates for a given action.

14.2 PPO versus SAC: score-function and pathwise gradients

PPO typically uses $\nabla_\theta \log \pi_\theta(a \mid s)\, \hat{A}(s,a)$ along with clipped likelihood ratios. SAC reparameterizes continuous actions so that $\nabla_a Q(s,a)\, \partial a / \partial \theta$ reaches the actor, together with an entropy gradient. Standard PPO does not require its critic to be differentiable with respect to actions; standard continuous SAC generally does. PPO can be fast in wall-clock time when simulators produce massively parallel fresh rollouts. SAC can be more sample-efficient when physical interactions are expensive. **Sample efficiency is not the same as wall-clock efficiency**.

15. Robot-control limits and common misconceptions

15.1 Why critic extrapolation is especially important in manipulation

Robot arms and dexterous hands involve mode-switching contacts, nonlinear dynamics, sparse rewards, estimation errors, and action saturation. If replay lacks some contact configurations or recovery motions, the critic may assign unrealistically high values to out-of-coverage actions. An actor that follows Q-gradients may then move toward precisely those spurious values, creating an **actor-critic feedback loop**. Twin critics mitigate part of this risk; they do not replace coverage analysis, safety constraints, reward checks, or monitoring of distribution shift.

15.2 Joint positions, velocities, torques, and action normalization

A policy action in $[-1,1]^d$ is often not itself the joint position, velocity, or torque sent to the robot. The control interface must apply a clear scale, offset, and physical limits. Saturation can reduce the derivative of tanh; scaling affects density and the entropy target. A correct MDP description should also account for control frequency, sensing latency, actuator bounds, and the chosen action representation. These details often matter more for trustworthy robot behavior than the elegance of the actor loss alone.

15.3 When standard SAC should not be copied verbatim

Standard SAC assumes actions can be sampled from a policy with an evaluable log density and usually exploits a differentiable path from actor parameters to actions. Discrete actions, implicit generative policies, flow-matching action heads, partial observability with long history, and strict safety constraints can require different objectives or parameterizations. It is invalid to attach $\alpha \log \pi$ to a policy whose density is unavailable and automatically call the result standard SAC. Likewise, replayed data do not by themselves resolve out-of-distribution action values in offline RL; **offline RL and online off-policy RL are different settings**.

15.4 Five misconceptions to avoid

1. "**Q-learning is off-policy only when a replay buffer is used.**" No: the distinction comes from behavior versus target policies.
2. "**Soft Q-learning is just Q-learning's max replaced by an actor sample.**" Incomplete: distinguish soft optimality from soft evaluation of a fixed policy.
3. "**Double Q-learning, Double DQN, and SAC min-Q are the same estimator.**" No: their selection/evaluation and estimator-combination rules differ.
4. "**Larger temperature always yields higher task return.**" No: entropy trades off with task reward and is sensitive to action and reward scales.
5. "**Off-policy reuse guarantees stable, safe learning.**" No: support mismatch, bootstrapping error, nonstationarity, and execution constraints still matter.

16. The complete derivation in six equations

The following six formulas summarize the bridge from classical Q-learning to modern SAC.

(1) Ordinary Bellman optimality:

$$\boxed{Q^*(s,a) = r(s,a) + \gamma \mathbb{E}_{s'}[\max_{a'} Q^*(s',a')].}$$

(2) Q-learning's sampled TD update:

$$\boxed{Q(s,a) \leftarrow Q(s,a) + \eta \left[r + \gamma(1-d) \max_{a'} Q(s',a') - Q(s,a) \right].}$$

(3) Optimal soft value for finite discrete actions:

$$\boxed{V^*_{\text{soft}}(s) = \alpha \log \sum_a \exp(Q^*_{\text{soft}}(s,a)/\alpha),}$$

whose zero-temperature limit recovers $\max_a Q(s,a)$.

(4) Soft Bellman evaluation of a given policy:

$$\boxed{Q^\pi_{\text{soft}}(s,a) = r + \gamma \mathbb{E}_{s',a' \sim P,\pi}[Q^\pi_{\text{soft}}(s',a') - \alpha \log \pi(a' \mid s')].}$$

(5) SAC's twin critics and reparameterized actor:

$$\boxed{\begin{aligned} y &= r + \gamma(1-d) \left[\min_j Q_{\bar{\phi}_j}(s',a') - \alpha \log \pi_\theta(a' \mid s') \right], \\ J_\pi &= \mathbb{E}_{s,\epsilon} \left[\alpha \log \pi_\theta(f_\theta(s,\epsilon) \mid s) - \min_j Q_{\phi_j}(s, f_\theta(s,\epsilon)) \right]. \end{aligned}}$$

(6) Entropy target and temperature adaptation:

$$\boxed{J^{\text{sur}}_\eta = -\mathbb{E}\left[\eta(\log \pi_\theta(a \mid s) + \mathcal{H}_{\text{target}}) \right], \quad \eta = \log \alpha.}$$

In one sentence: Q-learning learns optimal action values using **hard Bellman optimality and TD bootstrapping**, whereas SAC combines **soft Bellman policy evaluation**, **actor policy improvement**, **and entropy adaptation** to learn a stochastic continuous-control policy. They share Bellman reasoning and TD learning; they differ in how future actions are optimized and how policy entropy is treated.

References and further reading

1. Watkins, C. J. C. H., & Dayan, P. (1992). **Q-learning**. *Machine Learning*, 8, 279–292. DOI. Classical tabular Q-learning and convergence analysis.
2. Sutton, R. S., & Barto, A. G. (2018). **Reinforcement Learning: An Introduction**, 2nd ed. Open textbook. Bellman equations, TD, SARSA, Q-learning, and policy iteration.
3. Mnih, V., et al. (2015). **Human-level control through deep reinforcement learning**. *Nature*. DOI. DQN, replay, and target networks.
4. van Hasselt, H. (2010). **Double Q-learning**. *NeurIPS*. Paper. Estimation bias and two-estimator learning.
5. van Hasselt, H., Guez, A., & Silver, D. (2016). **Deep Reinforcement Learning with Double Q-learning**. arXiv:1509.06461. Double DQN.
6. Lillicrap, T. P., et al. (2015). **Continuous control with deep reinforcement learning**. arXiv:1509.02971. Deterministic continuous actor-critic.
7. Fujimoto, S., van Hoof, H., & Meger, D. (2018). **Addressing Function Approximation Error in Actor-Critic Methods**. arXiv:1802.09477. TD3 and twin critics.

8. Haarnoja, T., Tang, H., Abbeel, P., & Levine, S. (2017). **Reinforcement Learning with Deep Energy-Based Policies**. *ICML*. Paper. Soft Q-learning and energy-based policies.
9. Haarnoja, T., Zhou, A., Abbeel, P., & Levine, S. (2018). **Soft Actor-Critic: Off-Policy Maximum Entropy Deep Reinforcement Learning with a Stochastic Actor**. arXiv:1801.01290. Soft policy iteration and KL improvement.
10. Haarnoja, T., et al. (2018). **Soft Actor-Critic Algorithms and Applications**. arXiv:1812.05905. Learned temperature and modern SAC.
11. OpenAI Spinning Up. **Soft Actor-Critic**. Algorithm guide. Practical loss definitions and implementation conventions.

License note: original course content is credited under CC BY 4.0; third-party cited works retain their own licenses. This teaching document includes no experiments, benchmark claims, or guarantees about training outcomes. Equations use standard LaTeX math compatible with GitHub Markdown and Pandoc. Its accompanying PDF is generated from this exact Markdown revision.